

Role-based access control

COMPLETE LEARNING GUIDE

What You'll Learn

Authentication answers 'who are you?'; authorization answers 'what are you allowed to do?' — and role-based access control (RBAC) is the most common pattern for implementing the second question at scale. Instead of checking permissions for each individual user directly, RBAC introduces roles (admin, editor, viewer) as a layer in between: permissions are granted to roles, and users are assigned to one or more roles, so managing access for a thousand users means managing a handful of roles instead of a thousand individual permission sets. This guide covers designing an RBAC data model, enforcing it consistently through server-side middleware (never trusting a client-side check alone), the distinction between coarse roles and fine-grained permissions, and why role checks alone are still not enough without also verifying resource ownership.

1. Authentication vs. Authorization — A Distinction Worth Repeating

These two terms get conflated constantly, but they're answering entirely different questions and require entirely different failure responses. Authentication establishes identity — did this request come with valid credentials at all — and its failure response is 401 Unauthorized. Authorization establishes permission — given that we know who this is, are they allowed to do this specific thing — and its failure response is 403 Forbidden. RBAC lives entirely on the authorization side: it assumes authentication has already succeeded and `req.user` is already populated, and its job is purely to decide whether that already-identified user can proceed.

2. The RBAC Data Model: Users, Roles, and Permissions

A flexible RBAC schema separates three concepts with many-to-many relationships between them: users can hold multiple roles, roles can hold multiple permissions, and permissions represent a specific allowed action (often named `resource:action`, like `orders:delete` or `users:invite`). This indirection is the entire point — granting a new permission to the 'editor' role instantly applies to every user with that role, without touching per-user records at all, which is what makes RBAC manageable at scale compared to assigning permissions directly to individual users.

SQL

```
CREATE TABLE roles (  
  role_id SERIAL PRIMARY KEY,  
  name VARCHAR(50) NOT NULL UNIQUE -- e.g. 'admin', 'editor', 'viewer'  
);  
  
CREATE TABLE user_roles (  
  user_id INTEGER NOT NULL REFERENCES users(user_id),  
  role_id INTEGER NOT NULL REFERENCES roles(role_id),  
  PRIMARY KEY (user_id, role_id)  
);  
  
CREATE TABLE permissions (  
  permission_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE -- e.g. 'orders:delete'  
);  
  
CREATE TABLE role_permissions (  
  role_id INTEGER NOT NULL REFERENCES roles(role_id),  
  permission_id INTEGER NOT NULL REFERENCES permissions(permission_id),  
  PRIMARY KEY (role_id, permission_id)  
);
```

3. Enforcing Roles in Middleware, Not in Every Handler

RBAC checks belong in a small, reusable piece of middleware placed between authentication and the route handler, not copy-pasted as an if-statement inside every controller function. A factory function that returns middleware parameterized by which role(s) are allowed keeps the check declarative and readable directly at the route definition, and guarantees every protected route enforces the rule the exact same way — a single bug fix in the middleware fixes every route that uses it at once.

JAVASCRIPT

```
function requireRole(...allowedRoles) {  
  return (req, res, next) => {  
    if (!req.user) {  
      return res.status(401).json({ error: "Not authenticated" });  
    }  
    if (!allowedRoles.includes(req.user.role)) {  
      return res.status(403).json({ error: "Insufficient permissions" });  
    }  
    next();  
  };  
}  
  
router.delete(  
  "/api/orders/:id",  
  requireAuth,  
  requireRole("admin"),  
  ordersController.remove  
);
```

4. Coarse Roles vs. Fine-Grained Permissions

A small number of broad roles (admin, editor, viewer) is simple to reason about but becomes limiting as an application grows — eventually someone needs 'can view orders but not delete them, and can also manage users' which doesn't map cleanly onto any single coarse role. Fine-grained permission checks (checking a specific permissions:orders:delete permission rather than a role name directly) decouple 'what a user can do' from 'what we happen to call the bundle of permissions they have,' letting an admin later create a new custom role combining permissions in a way the original three roles never anticipated, without any code changes.

- Coarse roles: simple, easy to reason about, but inflexible as requirements grow beyond the original role set.
- Fine-grained permissions: checked by name (orders:delete), assigned to roles in the database — new roles can be composed without code changes.
- Attribute-Based Access Control (ABAC) goes further still, deciding access based on runtime attributes (time of day, resource state, department) rather than fixed roles.

5. Roles Alone Aren't Enough: Ownership and Resource-Level Checks

A role check confirms a user's general capability ("editors can edit orders") but says nothing about which specific orders they should be allowed to edit — a role check alone would let any editor edit any order, including ones belonging to a completely different customer or team. Real authorization logic in a multi-tenant or multi-user system almost always needs a second check beyond the role: does this specific user own, or have been granted access to, this specific resource instance. This ownership check typically requires loading the resource first and comparing a field like resource.ownerId against req.user.id, which is why it usually lives in the controller/service layer rather than in generic role middleware.

6. Default Deny: The Safest Starting Posture

A secure authorization system should default to denying access and require an explicit grant, rather than defaulting to allow and requiring an explicit deny. In practice this means a newly added route with no RBAC middleware attached at all should ideally fail a security review, not silently be reachable by any authenticated (or worse, any unauthenticated) user just because nobody remembered to add a check. Some frameworks enforce this structurally by requiring every route to declare its required permission explicitly; in frameworks that don't, this has to be enforced by code review discipline and, ideally, an automated test that walks every route and asserts it has an authorization check.

7. Testing Authorization Logic Directly

Authorization bugs are a common source of real-world security incidents precisely because they're easy to miss in casual manual testing — a developer testing as an admin will never notice that a regular user can also reach an admin-only route, because they never tried it as a regular user. Authorization logic deserves its own explicit automated tests: for each protected route, a test asserting that a user without the required role gets a 403, that a user without the required ownership gets a 403 (not just a role check), and that an unauthenticated request gets

a 401 — run these on every change to authorization code, not just once at launch.

Common Pitfalls

- Enforcing role checks only in the frontend UI (hiding a button) and trusting the client, without any corresponding check on the server.
- Scattering ad-hoc role-check if-statements across individual controllers instead of centralizing them in reusable middleware.
- Checking a user's role but never checking resource ownership, letting any user with the right role act on resources they shouldn't have access to.
- Confusing 401 and 403 in authorization failures, making it unclear to the client whether logging in again would help.
- Adding a new route without any authorization middleware and assuming it's 'fine for now' — an easy way for an unprotected endpoint to reach production.
- Never writing automated tests for authorization logic, so a permission bug is only discovered after being exploited, not during code review.

Key Takeaways

- Authentication (401, who are you) and authorization (403, what can you do) are distinct concerns with distinct failure codes.
- RBAC's core value is indirection: assign permissions to roles, assign roles to users, and manage access for many users by editing a handful of roles.
- Centralize role/permission checks in reusable middleware rather than duplicating if-statements across every controller.
- A role check alone is not enough for resource-specific actions — always also verify the requesting user actually owns or has been granted the specific resource.
- Default to deny: a route with no authorization check attached should be treated as a bug, not an oversight to fix later.
- Write explicit automated tests for authorization logic — it's one of the highest-value places for tests, precisely because manual testing tends to miss it.

Learn More

OWASP — Access Control Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

NIST — Role Based Access Control

<https://csrc.nist.gov/projects/role-based-access-control>

Auth0 — Role-Based Access Control (RBAC)

<https://auth0.com/docs/manage-users/access-control/rbac>